

S.E.P.

S.E.S.

TecNM



EDUCACIÓN
SECRETARÍA DE EDUCACIÓN PÚBLICA



TECNOLÓGICO
NACIONAL DE MÉXICO



Instituto Tecnológico de Aguascalientes
Tecnologías de la Información y la Comunicación

ACTIVIDAD

Wifi 1 _ESP32

Tecnologías Inalámbricas

Maestro: RICARDO ALEJANDRO RODRIGUEZ JIMENEZ

Alumnos:

Casas Demetrio Jorge Alberto
Zacarias Garcia Jason Alexander

Fecha:18 de mayo de 2026

Contenido

Introducción.....	3
Link de video.....	4
Link de GitHub	4
Código	5
Evidencias	14
Conclusión.....	15

Introducción

En los proyectos de Internet de las Cosas (IoT), uno de los principales retos es conectar los dispositivos a una red Wi-Fi de forma sencilla y segura. En muchos casos, no es práctico configurar manualmente los datos de conexión desde el código, especialmente cuando los dispositivos se instalan en distintos lugares.

Para resolver este problema, en este proyecto desarrollamos un sistema de configuración inalámbrica usando un ESP32 y un Portal Cautivo. Al encenderse, el dispositivo crea su propia red Wi-Fi y muestra automáticamente una página de configuración a la que el usuario puede acceder desde su celular o computadora.

En esta página, el usuario ingresa los datos necesarios, como el nombre de identificación del dispositivo y las credenciales de la red Wi-Fi a la que se conectará. Después, el sistema procesa la información y la envía al servidor principal para completar la configuración.

Con este método, se facilita la instalación de dispositivos inteligentes sin necesidad de cables, programas adicionales o conocimientos técnicos avanzados, haciendo más práctico y escalable el uso de tecnología IoT.

Link de video

https://youtube.com/shorts/sRikOqgh5OY?si=PAj3aEVi_uAwq1vI

Link de GitHub

https://github.com/JasonMakana/WiFi_ESP32.git

Código

// LIBRERÍAS

```
#include <WiFi.h>      // Controla las funciones de radio Wi-Fi del ESP32 (Modos AP, STA y Dual)
#include <DNSServer.h>  // Servidor DNS para interceptar peticiones del celular (Portal Cautivo)
#include <WebServer.h>  // Permite crear un servidor HTTP local para servir el formulario de configuración
#include <HTTPClient.h>
#include <ArduinoJson.h> // Maneja el parseo, lectura y empaquetado de datos en formato JSON
```

// INSTANCIAS Y CONFIGURACIÓN DE RED LOCAL

```
DNSServer dnsServer;  // Instancia para controlar el redireccionamiento DNS
WebServer server(80);  // Instancia del servidor Web escuchando en el puerto 80
```

```
const byte DNS_PORT = 53;  // Puerto estándar de peticiones DNS
IPAddress apIP(192, 168, 4, 1); // Dirección IP estática que tendrá el ESP32 como Access Point
```

// VARIABLES DE CONECTIVIDAD Y ENDPOINTS (BACKEND)

```
const char* ap_ssid = "LinkHealth_AP"; // Nombre de la red Wi-Fi que emitirá
```

```
String IpServer = "http://192.168.1.7:8000"; // Dirección base del servidor Laravel (Modificar según la IP de red)
String serverAccess = IpServer + "/api/esp32/acceso"; // API de Laravel encargado de registrar/actualizar el dispositivo
```

// VARIABLES DE TIEMPO PARA MANTENER LA CONEXIÓN ACTIVA

```
unsigned long previousMillis = 0;
const long interval = 30000; // Revision del Wi-Fi cada 30 segundos
```

// PORTAL CAUTIVO EN FORMULARIO HTML

```
const char PORTAL_HTML[] PROGMEM = R"=====(
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>LinkHealth Portal</title>
```

```

<!-- Fuerza al navegador del celular a renderizar en tamaño adaptativo móvil -->
<meta name="viewport" content="width=device-width, initial-scale=1">
<style>
  body { font-family: sans-serif; background: #f4f7f6; text-align: center; padding:
20px; }
  .card { background: white; padding: 20px; border-radius: 10px; box-shadow: 0 2px
5px rgba(0,0,0,0.1); }
  input { width: 90%; padding: 12px; margin: 8px 0; border: 1px solid #ddd; }
  button { background: #007bff; color: white; border: none; padding: 15px; width:
100%; cursor: pointer; }
</style>
</head>
<body>
<div class="card">
  <h2>Bienvenido a LinkHealth</h2>
  <p>Introduce los datos para activar el asistente auxiliar.</p>

  <!-- El formulario envía una petición POST local a la ruta /save mapeada en el ESP32
-->
  <form action="/save" method="POST">
    <!-- Los atributos 'name' son las claves que recuperará server.arg() en handleSave
-->
    <input name="paciente" placeholder="Nombre del Paciente" required>
    <input name="clinica" placeholder="ID de Clínica" required>
    <hr>
    <input name="ssid" placeholder="Nombre Wi-Fi (SSID)" required>
    <input name="pass" type="password" placeholder="Password Wi-Fi">
    <button type="submit">CONFIGURAR Y CONECTAR</button>
  </form>
</div>
</body>
</html>
)====";

```

```

void handleSave() {
  // Extracción de los argumentos enviados por el Portal Cautivo
  // server.arg("name") intercepta lo que el usuario escribió en cada input
  String p = server.arg("paciente");
  String c = server.arg("clinica");
  String s = server.arg("ssid");
  String pass = server.arg("pass");

  Serial.println("Conectando a WiFi para reportar a Laravel...");

```

```

// Intento de conexión del ESP32 a la red Wi-Fi dada por el usuario
// .c_str() convierte los Strings de Arduino a arreglos de caracteres (char*) requeridos
por el SDK
WiFi.begin(s.c_str(), pass.c_str());

// Bucle de espera (Timeout de 10 segundos)
int tries = 0;
while (WiFi.status() != WL_CONNECTED && tries < 20) {
    delay(500);
    Serial.print(".");
    tries++;
}

// Evaluación del estado de la conexión Wi-Fi
if (WiFi.status() == WL_CONNECTED) {
    // ÉXITO: El ESP32 ya tiene acceso a la red local e Internet

    // Obtenemos la MAC física real una vez que la antena Wi-Fi está activa
    String mac = WiFi.macAddress();

    HTTPClient http;

    // Inicializa la petición HTTP con la URL del API (/api/esp32/acceso)
    http.begin(serverAccess);

    // Añadir cabecera obligatoria para indicar que enviaremos un cuerpo en formato
JSON
    http.addHeader("Content-Type", "application/json");

    // Construcción manual y limpia de la cadena JSON (String)
    String json = "{}";
    json += "\"mac_address\": \"" + mac + "\"";
    json += "\"paciente_name\": \"" + p + "\"";
    json += "\"clinica_id\": \"" + c + "\"";
    json += "\"ssid\": \"" + s + "\"";
    json += "\"password\": \"" + pass + "\"";
    json += "}";

    // Enviando del JSON armado
    int httpResponseCode = http.POST(json); // Envía los datos y recupera el código
de estado HTTP
    String response = http.getString(); // Captura el JSON de respuesta enviado por
Laravel

```

```

// Muestra del JSON resivido de Laravel
Serial.print("Código HTTP: ");
Serial.println(httpResponseCode);
Serial.print("Respuesta: ");
Serial.println(response);

// Evaluación de la respuesta del servidor Laravel
if (httpResponseCode > 0) {
    // Texto muestra en Portal Cautivo
    server.send(200, "text/html", "<h1>Exito</h1><p>Vinculado correctamente.</p>");
    delay(3000); // Breve espera para asegurar que el navegador reciba la
confirmación de Éxito

    // Reinicio de software para limpiar la pila de red y arrancar en modo operativo
(STA)
    ESP.restart();
} else {
    // Error de red del lado del cliente o caída del servidor (Códigos negativos como -1)
    server.send(200, "text/html", "<h1>Error</h1><p>No se pudo contactar al
servidor.</p>");
}
http.end(); // Cierra la conexión HTTP y libera la memoria asignada a la petición
} else {
    // FALLA: Las credenciales del Wi-Fi proporcionadas por el usuario fueron
incorrectas
    server.send(200, "text/html", "<h1>Error</h1><p>WiFi incorrecto.</p>");
}
}

void verificarRegistroEnLaravel() {
    HTTPClient http;
    String mac = WiFi.macAddress(); // Obtiene la MAC física real del hardware

    // Construcción de la URL dinámica inyectando la MAC como parámetro de ruta GET
    String url = IpServer + "/api/esp32/credenciales/" + mac;

    Serial.println("Consultando URL: " + url);

    // Inicializa la petición HTTP con el API de consulta
    http.begin(url);

    // Realiza una petición GET bloqueante y recupera el código de respuesta (ej. 200,
404, 500)
    int httpCode = http.GET();

```



```

// CASO EXITOSO: El dispositivo está registrado en la BD de Laravel
if (httpCode == 200) {
    String payload = http.getString(); // Captura el JSON plano enviado por el backend
    Serial.println("Respuesta de Laravel: " + payload);

    // Instancia del documento de ArduinoJson para alojar el objeto parseado
    JsonDocument doc;

    // Deserializa (parsea) la cadena JSON para convertirla en un objeto accesible por
    // claves
    DeserializationError error = deserializeJson(doc, payload);

    // Si el JSON se parseó correctamente y no está corrupto
    if (!error) {
        bool registrado = doc["registered"]; // Extrae la bandera booleana de registro

        if (registrado) {
            // Extrae los punteros de caracteres a las credenciales guardadas en la nube
            const char* ssid_db = doc["ssid"];
            const char* pass_db = doc["password"];

            Serial.printf("Credenciales de BD -> SSID: %s\n", ssid_db);

            // COMPARACIÓN CRÍTICA: Revisa si el Wi-Fi de la BD es diferente al Wi-Fi actual
            // del chip
            if (String(ssid_db) != WiFi.SSID()) {
                Serial.println("¡Nueva red detectada en BD! Aplicando cambios...");

                // Sobrescribe las credenciales internas e intenta la conexión a la nueva red
                WiFi.begin(ssid_db, pass_db);
                delay(2000);

                // Forzar un reinicio por software para que el ESP32 arranque de forma limpia
                // aplicando el modo STA directo a la nueva red Wi-Fi en el próximo inicio
                ESP.restart();
            } else {
                // El dispositivo y la base de datos están perfectamente sincronizados en la
                // misma red
                Serial.println("El dispositivo ya usa la red correcta de la BD.");
            }
        }
    }
}

```

```

// CASO CONTROLADO DE ERROR (404): El dispositivo fue borrado o no existe en
Laravel
// Esto cumple con la función de soporte/asistente técnico del firmware
else if (httpCode == 404) {
    Serial.println("\n[ALERTA] El dispositivo no existe en Laravel (404).");
    Serial.println("Limpiando credenciales y forzando Portal Cautivo...");

    // El primer parámetro borra las credenciales de la Flash; el segundo apaga el Wi-Fi
    actual
    WiFi.disconnect(true, true);
    delay(1000);

    // Se reactiva el servidor DNS secuestrador en el puerto 53 para atrapar el tráfico
    del usuario
    dnsServer.start(DNS_PORT, "*", apIP);
    Serial.println("Portal Cautivo forzado. Conéctate a LinkHealth_AP para registrar.");
}
// OTROS ERRORES
else {
    Serial.printf("Error de comunicación con Laravel. Código HTTP: %d\n", httpCode);
}

// Libera los recursos asignados a la conexión del cliente HTTP
http.end();
}

void iniciarPortalConfiguracion() {
    // Configurar el modo a únicamente Access Point (Apaga el cliente STA
    temporalmente)
    WiFi.mode(WIFI_AP);

    // Aplica la IP estática y la máscara de subred para la interfaz del AP local
    WiFi.softAPConfig(apIP, apIP, IPAddress(255, 255, 255, 0));

    // Despliega la red Wi-Fi abierta sin contraseña usando el SSID configurado
    WiFi.softAP(ap_ssid);

    // Infomración para nosotros de verificación
    Serial.println("--- MODO CONFIGURACIÓN ACTIVADO ---");
    Serial.print("Red: "); Serial.println(ap_ssid);
    Serial.print("IP del Portal: "); Serial.println(WiFi.softAPIP());

    // Iniciar el Servidor DNS en el puerto 53, "*" intercepta absolutamente cualquier
    petición de dominio

```

```

dnsServer.start(DNS_PORT, "*", apIP);

// Cuando accede a la raíz del servidor muestra el Portal Cautivo
server.on("/", []) {
  server.send(200, "text/html", PORTAL_HTML);
};

// Enrutamiento POST para procesar el formulario de guardado
server.on("/save", HTTP_POST, handleSave);

// ENRUTAMIENTO AUXILIAR PARA PORTALES CAUTIVOS:
// Los sistemas operativos Android e iOS envían peticiones ocultas a estos
endpoints para verificar
// si la red Wi-Fi tiene salida real a internet. Si el ESP32 responde con contenido a
estas rutas,
// el celular se da cuenta de que está atrapado e inmediatamente despliega la
ventana del portal.
server.on("/generate_204", []) { server.send(200, "text/html", PORTAL_HTML); }; //
Usado por Android/Chrome
server.on("/fwlink", []) { server.send(200, "text/html", PORTAL_HTML); }; // Usado
por Windows/Windows Phone

// Cualquier petición que no coincida con las rutas previas es redirigida al HTML de
configuración
server.onNotFound([]) {
  server.send(200, "text/html", PORTAL_HTML);
};

// 4. Encender el demonio del servidor HTTP local
server.begin();
Serial.println("Servidor Web del Portal iniciado.");
}

void setup() {
  Serial.begin(115200);
  delay(1000); // Pausa de seguridad para estabilizar los niveles de voltaje en el
módulo ESP32S3

  // Modo de operación híbrido/dual (Access Point + Station)
  // La ESP32 usa el internet de mi router y muestra su propia señal
  WiFi.mode(WIFI_AP_STA);

  // Inicializar inmediatamente la interfaz inalámbrica propia local
  WiFi.softAPConfig(apIP, apIP, IPAddress(255, 255, 255, 0));

```

```

WiFi.softAP(ap_ssid);
Serial.println("Red AP Activa y Visible: " + String(ap_ssid));

// Registro global de las rutas web
server.on("/", []() { server.send(200, "text/html", PORTAL_HTML); });
server.on("/save", HTTP_POST, handleSave);
server.on("/generate_204", []() { server.send(200, "text/html", PORTAL_HTML); });
server.on("/fwlink", []() { server.send(200, "text/html", PORTAL_HTML); });
server.onNotFound([]() { server.send(200, "text/html", PORTAL_HTML); });

// Intenta conexión automática con las credenciales cacheadas de la memoria NVS
flash
Serial.println("Intentando conectar a la red guardada en memoria...");
WiFi.begin();

// Bucle de espera para evaluar el estado de la antena por un límite máximo de 8
segundos
int checkTries = 0;
while (WiFi.status() != WL_CONNECTED && checkTries < 16) {
    delay(500);
    Serial.print(".");
    checkTries++;
}

// Evaluamos el resultado del intento de conexión automática
if (WiFi.status() == WL_CONNECTED) {
    Serial.println("\n[ESTADO] Conectado a Internet.");
    Serial.print("IP asignada por el router: "); Serial.println(WiFi.localIP());

    // Si el enlace fue exitoso, ejecutamos de fondo la verificación de la MAC en Laravel
    verificarRegistroEnLaravel();
} else {
    // Si no había credenciales válidas o el módem está apagado, activamos el
    redireccionador DNS
    Serial.println("\n[ESTADO] No se pudo conectar a internet automáticamente.");
    Serial.println("Activando DNS para Portal Cautivo...");

    // Se enciende el servidor de nombres DNS únicamente en este bloque para evitar
    secuestrar el tráfico
    // si el dispositivo ya cuenta con salida real a la WAN
    dnsServer.start(DNS_PORT, "*", apIP);
}

// Encender el servidor web local del dispositivo

```

```

server.begin();
Serial.println("Servidor HTTP del ESP32 iniciado correctamente.");
}

void loop() {
    // Escucha, procesa y atiende las peticiones entrantes de los clientes HTTP
    (celulares)
    server.handleClient();

    // Si la conexión Wi-Fi de la estación no está activa, procesa las peticiones de
    resolución DNS
    if (WiFi.status() != WL_CONNECTED) {
        dnsServer.processNextRequest();
    }

    // LÓGICA PARA MANTENER ACTIVA LA RED (KEEP-ALIVE)
    // Captura el tiempo transcurrido desde que se encendió el microcontrolador
    unsigned long currentMillis = millis();

    if (currentMillis - previousMillis >= interval) {
        previousMillis = currentMillis; // Actualiza la estampa de tiempo al ciclo actual

        // Si detectamos que la conexión a Internet se ha caído
        if (WiFi.status() != WL_CONNECTED) {
            Serial.println("[AUXILIAR] Se perdió la conexión a Internet. Intentando
            reconectar...");

            // Fuerza la desasociación limpia del punto de acceso anterior e intenta un re-
            enlace
            // de fondo utilizando las últimas credenciales almacenadas en el hardware
            WiFi.disconnect();
            WiFi.reconnect();
        } else {
            // Conexión estable. Espacio libre para ejecutar la lógica de LinkHealth de tu
            proyecto
            Serial.println("[AUXILIAR] Conexión activa y saludable.");

            // Para futuros trabajos, aquí coloca la lectura del lector NFC, sensores y envío
            periódico a Laravel

        }
    }
}

```

Evidencias

Introduce los datos para activar el asistente auxiliar.

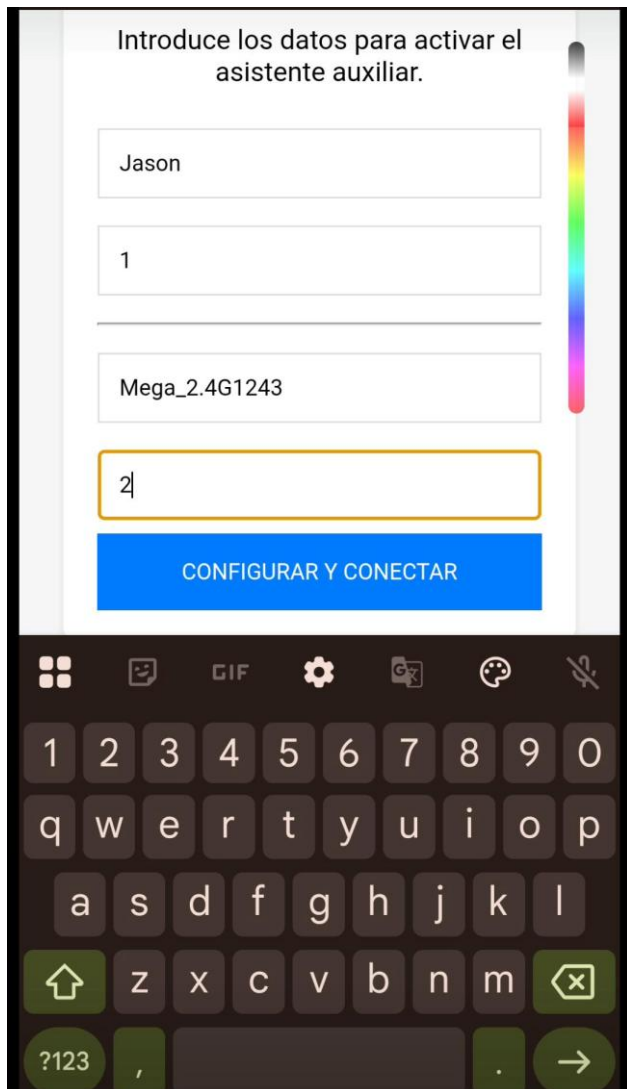
Jason

1

Mega_2.4G1243

2

CONFIGURAR Y CONECTAR



12:45

Sign in to LinkHealth_AP
connectivitycheck.gstatic.com

Bienvenido a LinkHealth

Introduce los datos para activar el asistente auxiliar.

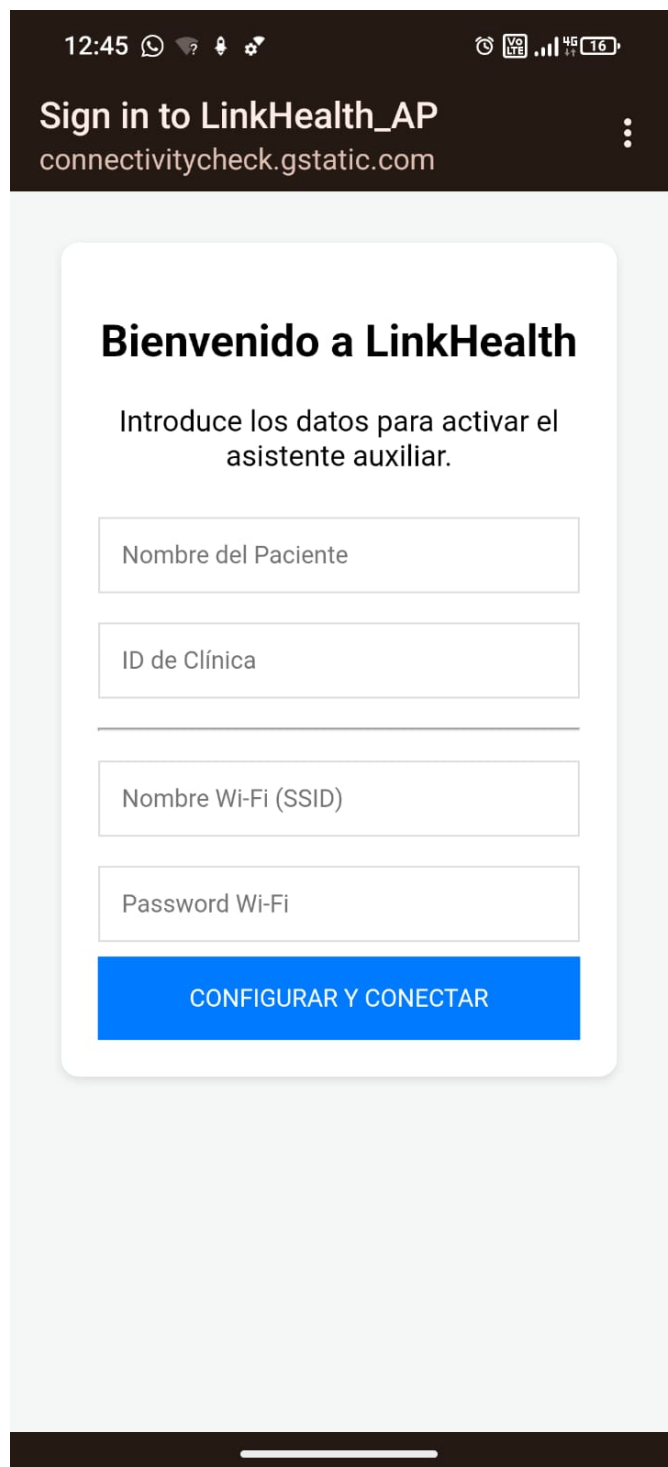
Nombre del Paciente

ID de Clínica

Nombre Wi-Fi (SSID)

Password Wi-Fi

CONFIGURAR Y CONECTAR



Conclusión

Zacarias García Jason Alexander

Esta práctica me permitió comprender mejor cómo funcionan las conexiones de red en dispositivos embebidos, especialmente el cambio entre los modos Access Point (AP) y Station (STA). También aprendo que un portal cautivo no solo consiste en mostrar una página web, sino en redirigir automáticamente al usuario hacia la pantalla de configuración para facilitar la conexión del dispositivo a una red Wi-Fi.

Este aprendizaje es importante para desarrollar dispositivos inteligentes que puedan configurarse de manera sencilla y automática, sin necesidad de modificar el código o realizar procesos técnicos complicados.

Casas Demetrio Jorge Alberto

Con la práctica, obtuve un poco de experiencia en el manejo de memoria y en el envío de información dentro de la ESP32. Aprendí a guardar la interfaz web en la memoria del dispositivo, recibir datos desde formularios y enviar la información al servidor en formato JSON.

Además, se comprobó que un microcontrolador como la ESP32 puede funcionar al mismo tiempo como una página web para el usuario y como un dispositivo capaz de comunicarse con sistemas externos, aprovechando de manera eficiente sus recursos.

.